

NextRAN-AI – 8/5/2026

Marco Bertuletti

mbertuletti@iis.ee.ethz.ch

Yichao Zhang

yiczhang@iis.ee.ethz.ch

Mahdi Abdollahpour

mahdi.abdollahpour@unibo.it

Alessandro Vanelli-Coralli

avanelli@iis.ee.ethz.ch

Luca Benini

lbenini@iis.ee.ethz.ch

PULP Platform

Open Source Hardware, the way it should be!



@pulp_platform 

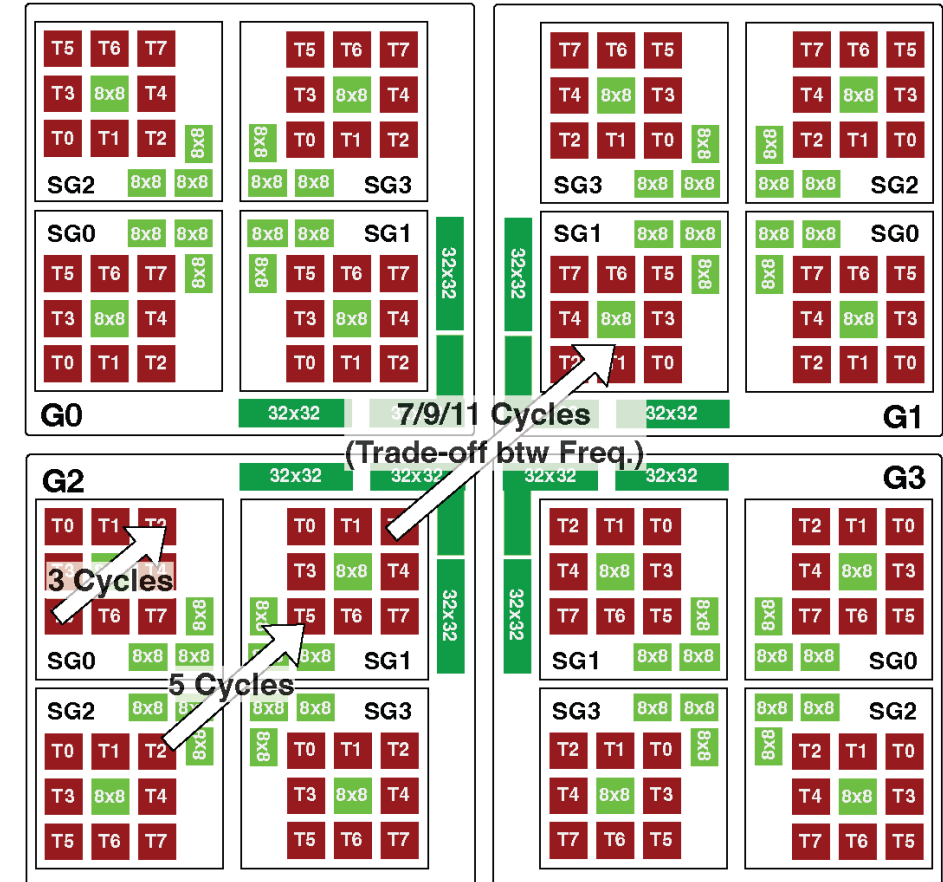
pulp-platform.org 

youtube.com/pulp_platform 

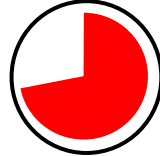
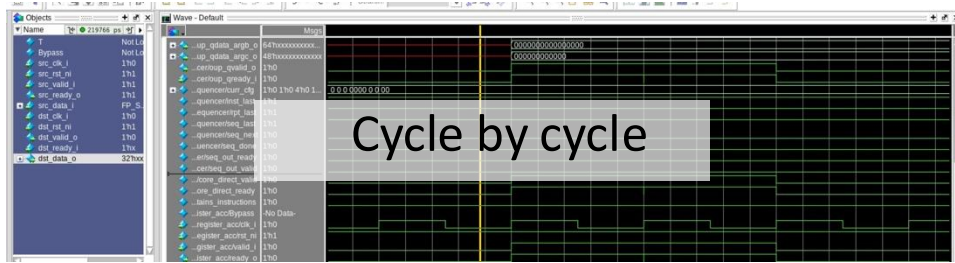
The largest cluster in PULP: TeraPool and TeraNoC



- **Up to 1024 Snitch cores and 4MiB unified L1 SPM**
 - Capability for modern workload
 - Ease of programming
- **Scalable: MinPool, MemPool and TeraPool**
- **TeraPool: Multi-level crossbar based L1 interconnect**
 - Low and deterministic access latency
 - Higher routing complexity as scaling up



Accuracy v.s. Simulation Runtime



- **Cycle-accurate simulator**

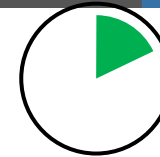
- High timing precision
- Long simulation runtime (days even weeks)

gvsoc/**gvsoc**



Event by event

2 Contributors 0 Issues 5 Stars 9 Forks

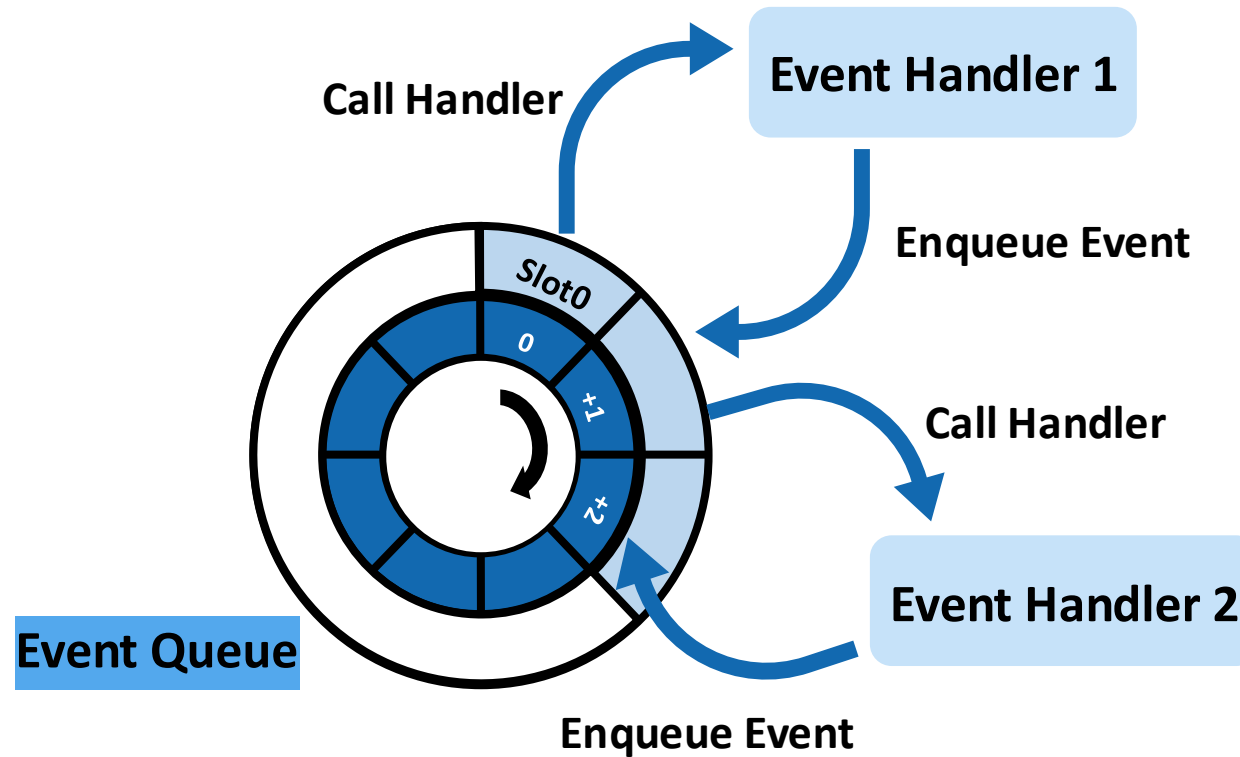


- **Event-based simulator**

- Timing-approximate
- Much faster (usually within hours)

- **Event-driven simulation model**

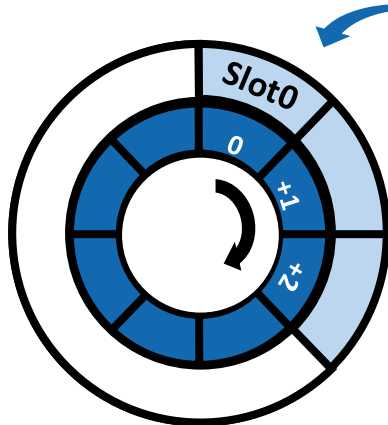
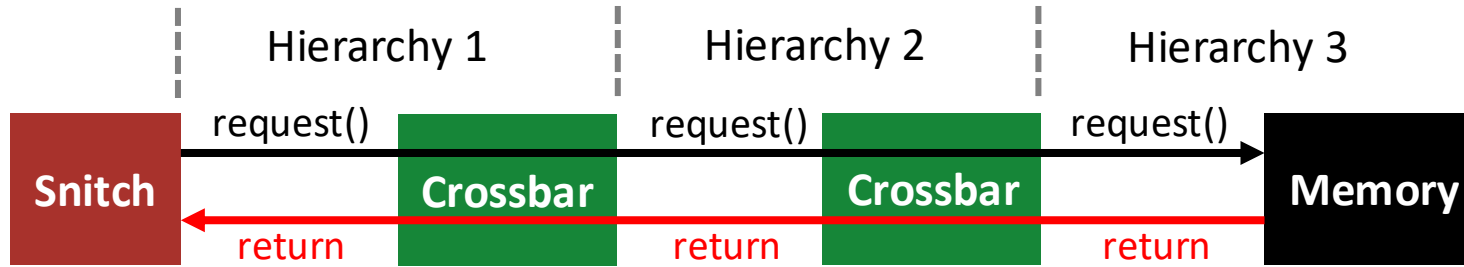
- Event queue + clock engine



Synchronous Modelling in GVSoC



- **Requests served within the request function call**
 - Can be forwarded with nested request call
 - Latency value accumulated along the path



Three hierarchies are handled within
Only **one** simulator event
Only **one** (nested) request call
Event Queue

Call stack:

L1.request(pkt)

processing...

pkt.latency++

L2.request(pkt)

processing...

pkt.latency++

L3.request(pkt)

processing...

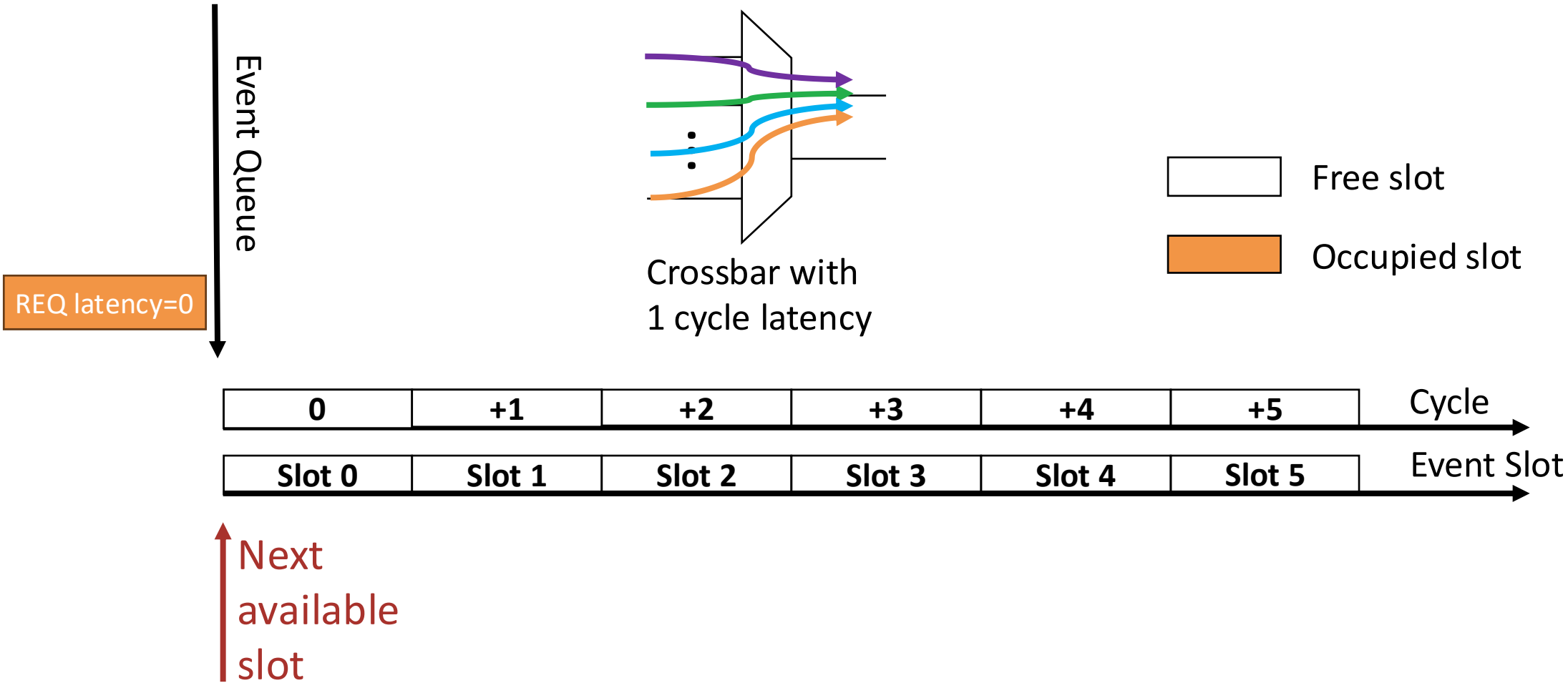
pkt.latency++

return

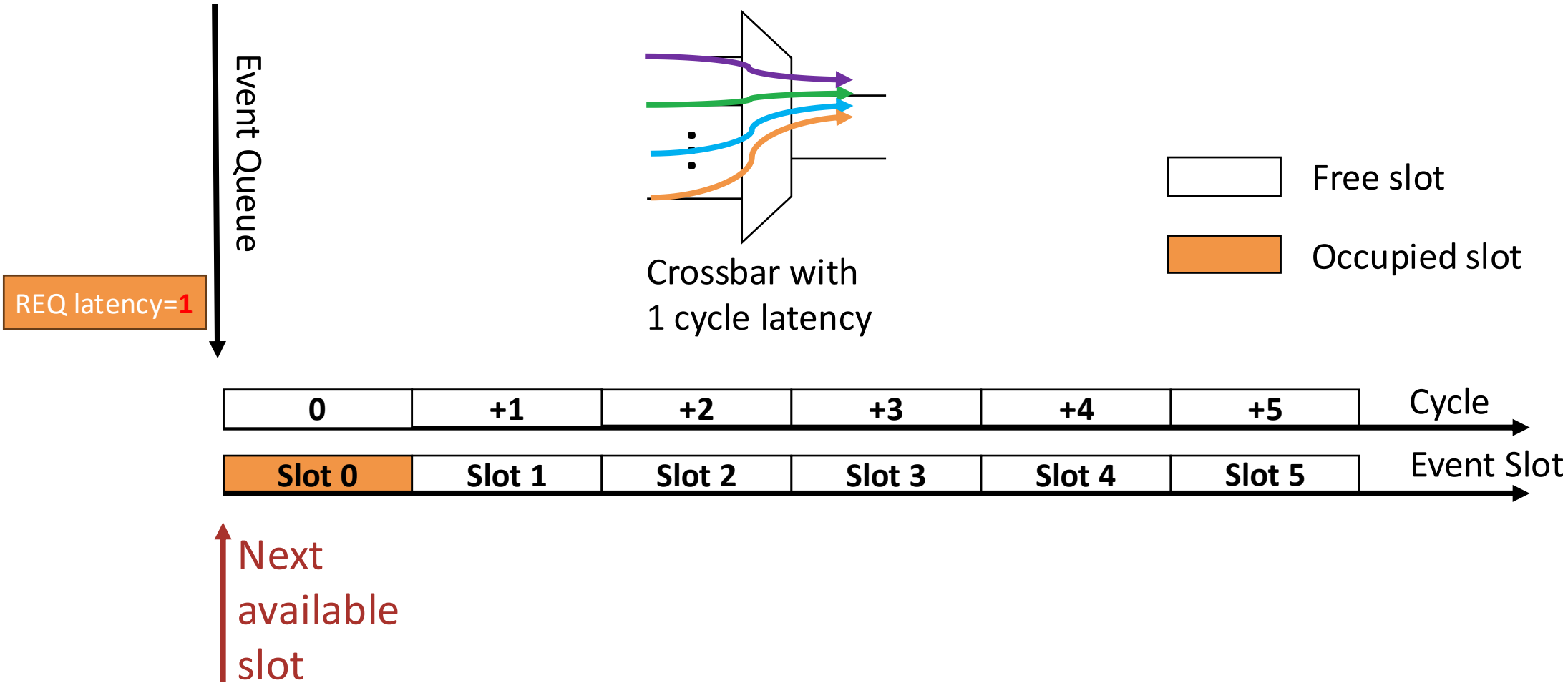
return

return

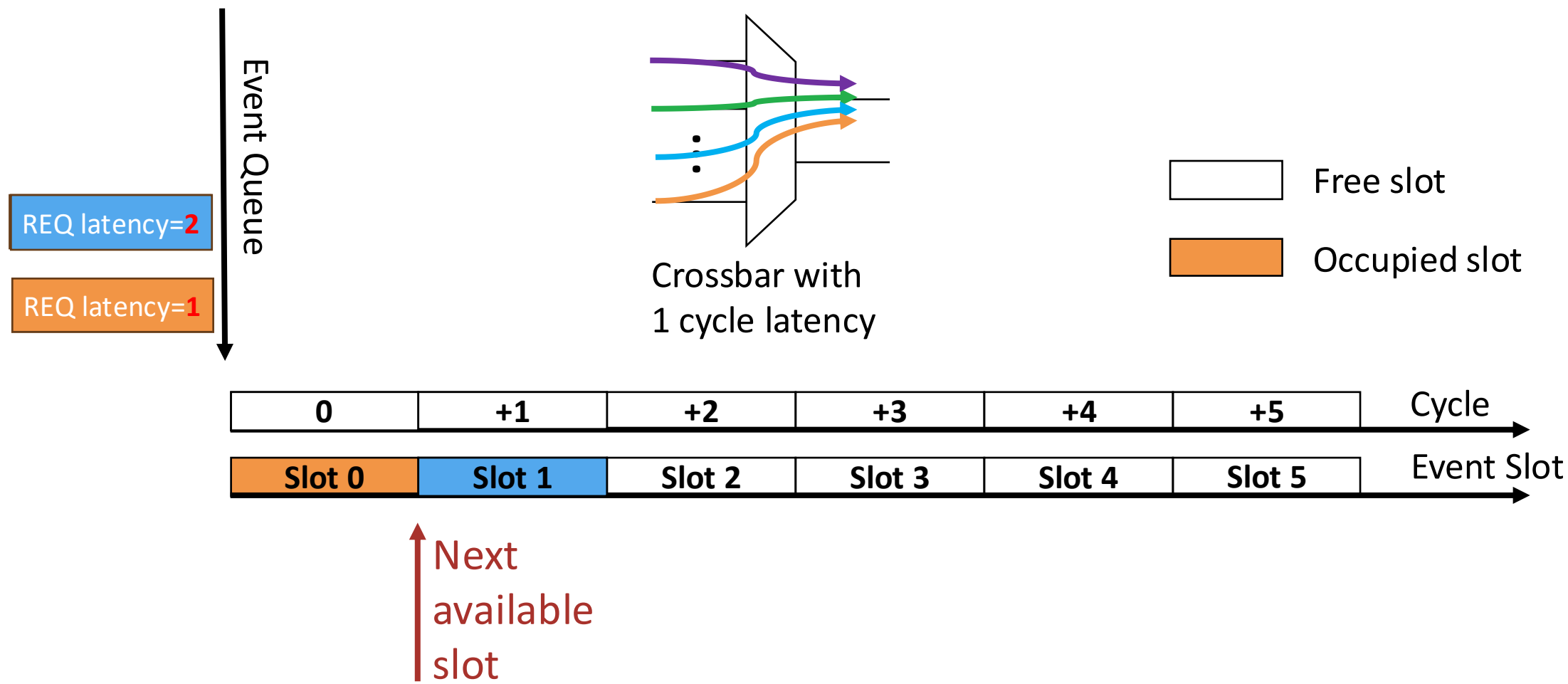
Synchronous Modelling in GVSoC



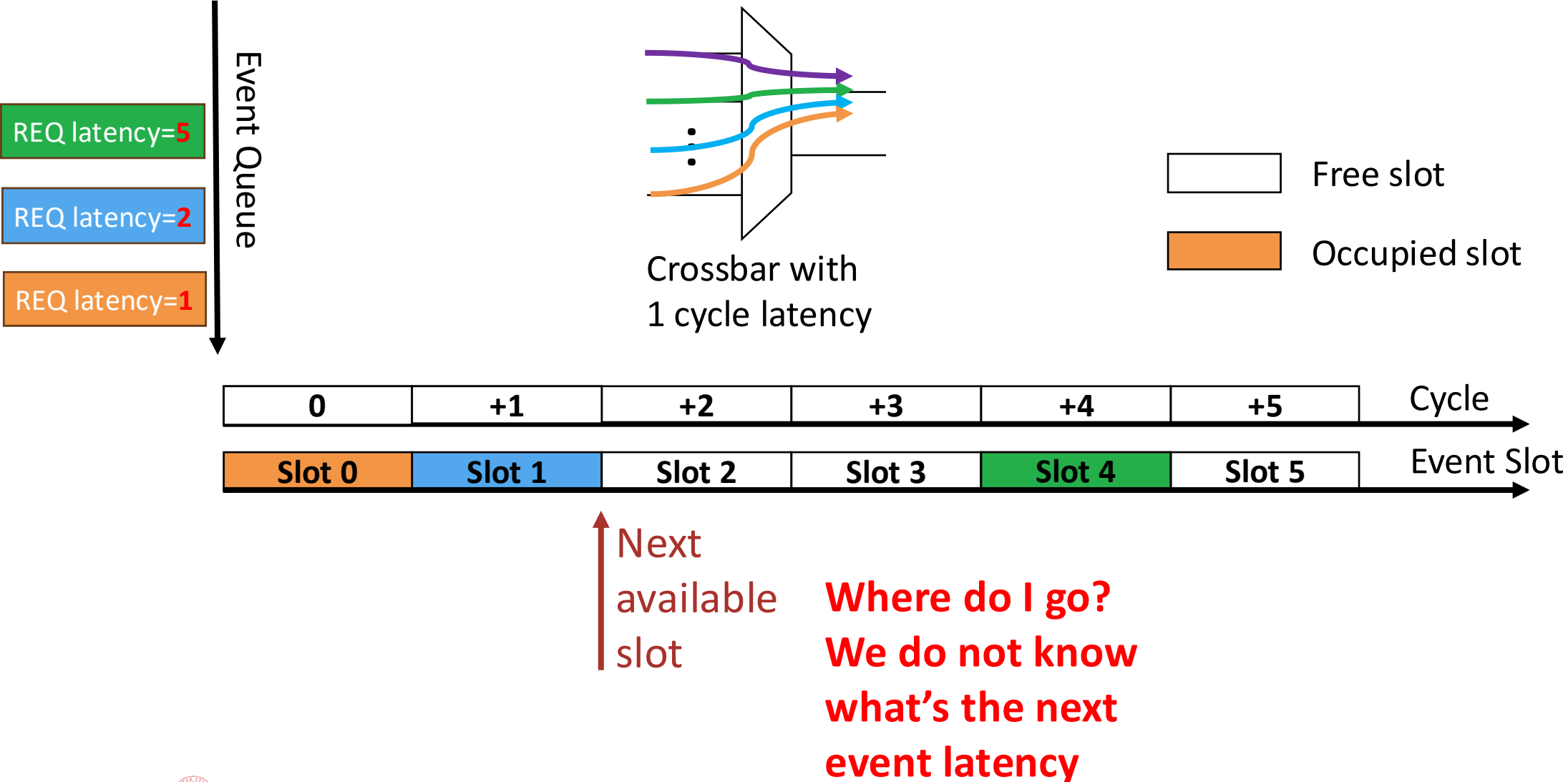
Synchronous Modelling in GVSoC



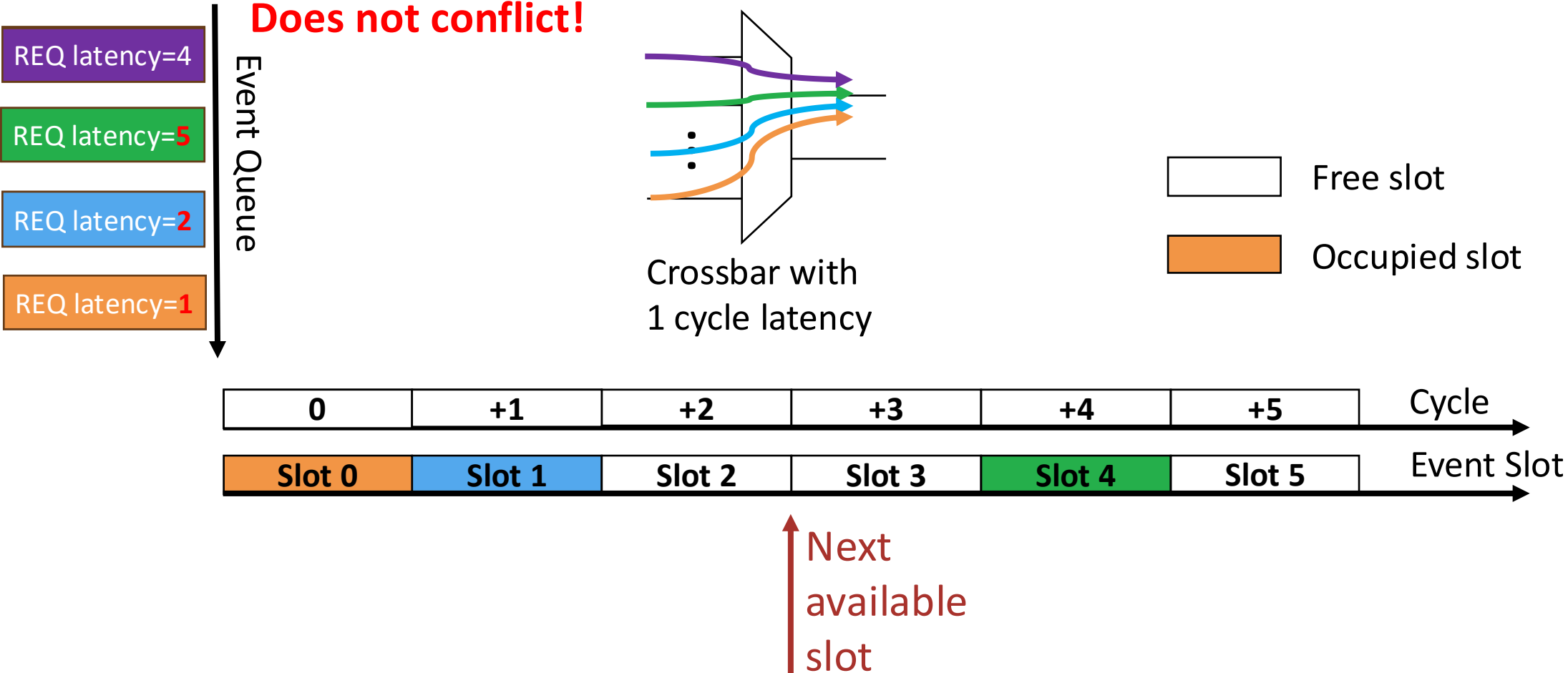
Synchronous Modelling in GVSoC



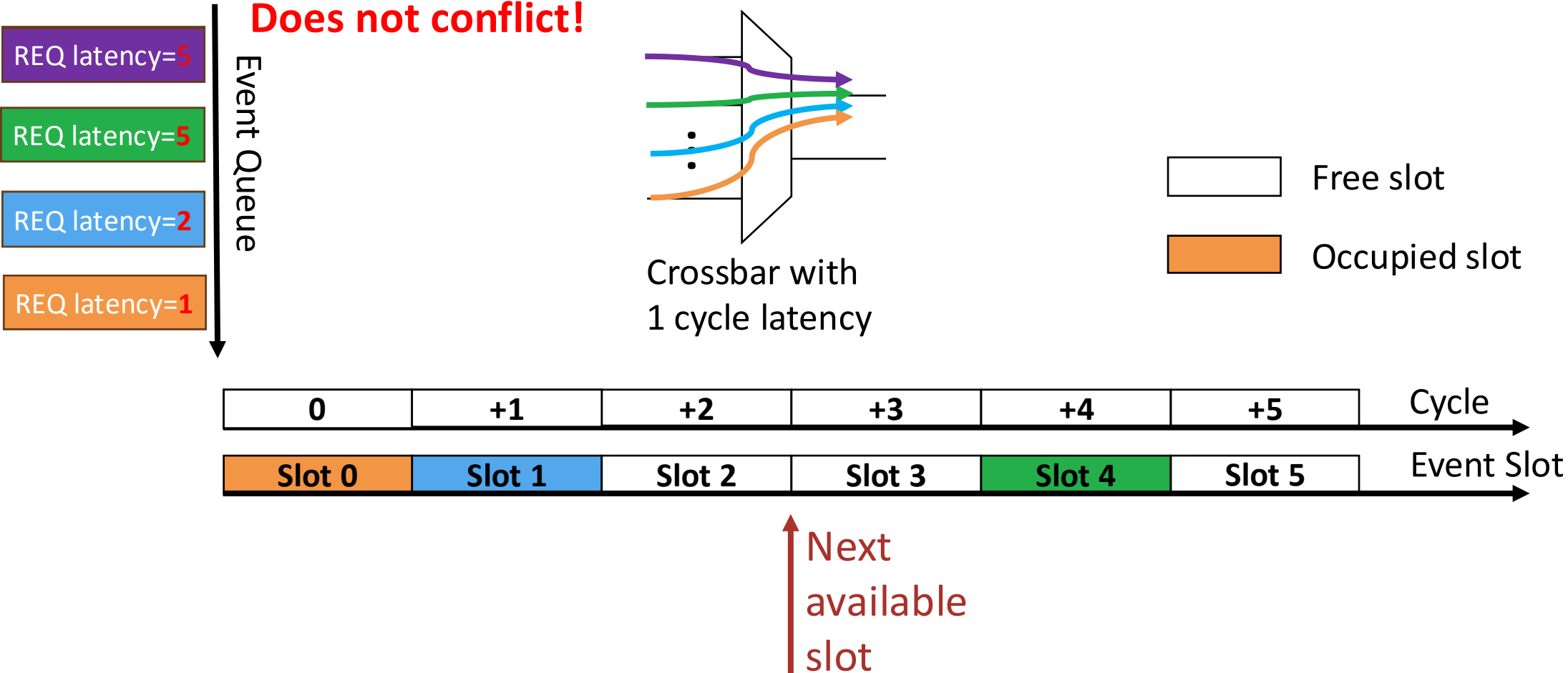
Synchronous Modelling in GVSoC



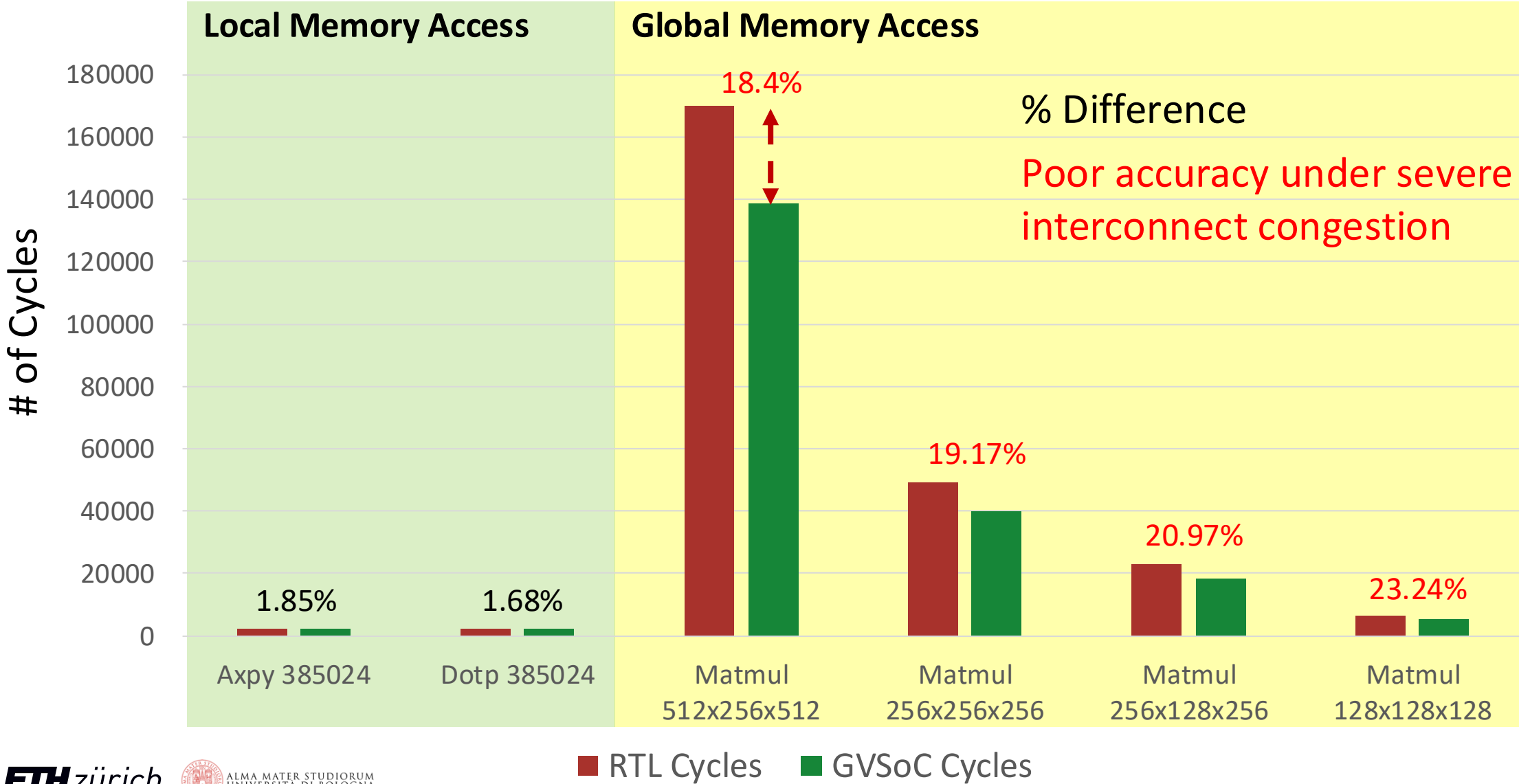
Synchronous Modelling in GVSoC



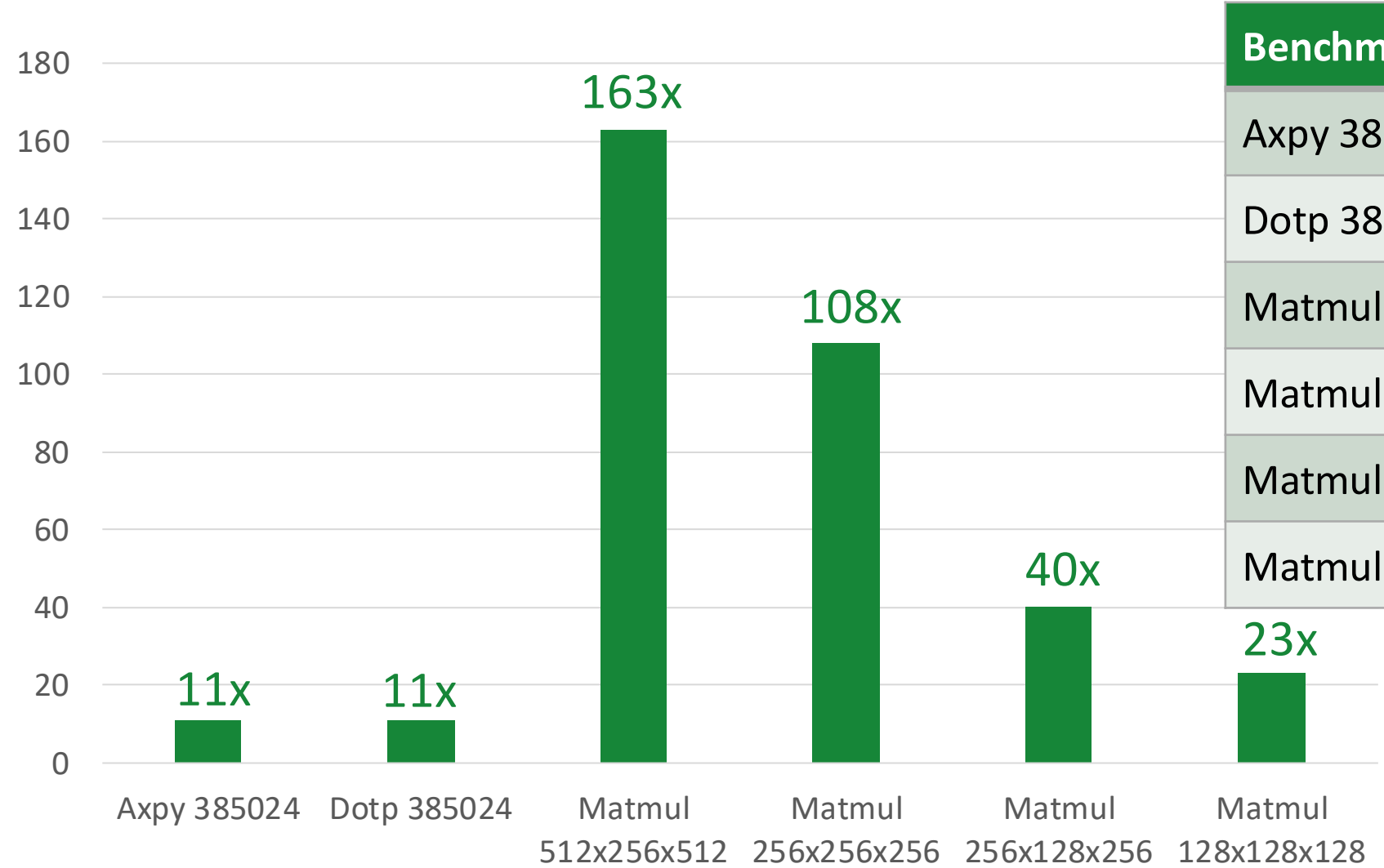
Synchronous Modelling in GVSoC



Synchronous Modelling in GVSoC



Speed up compared with RTL simulation



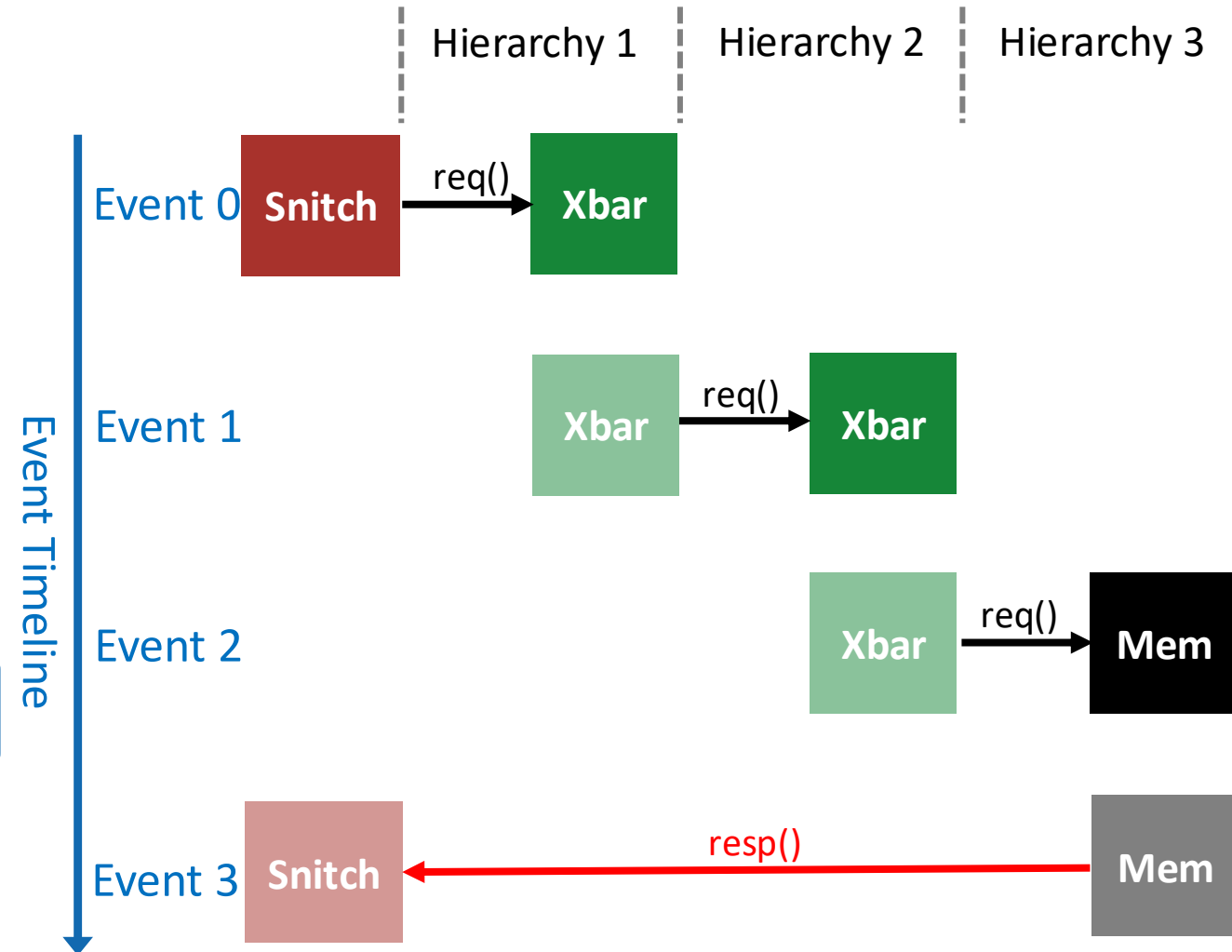
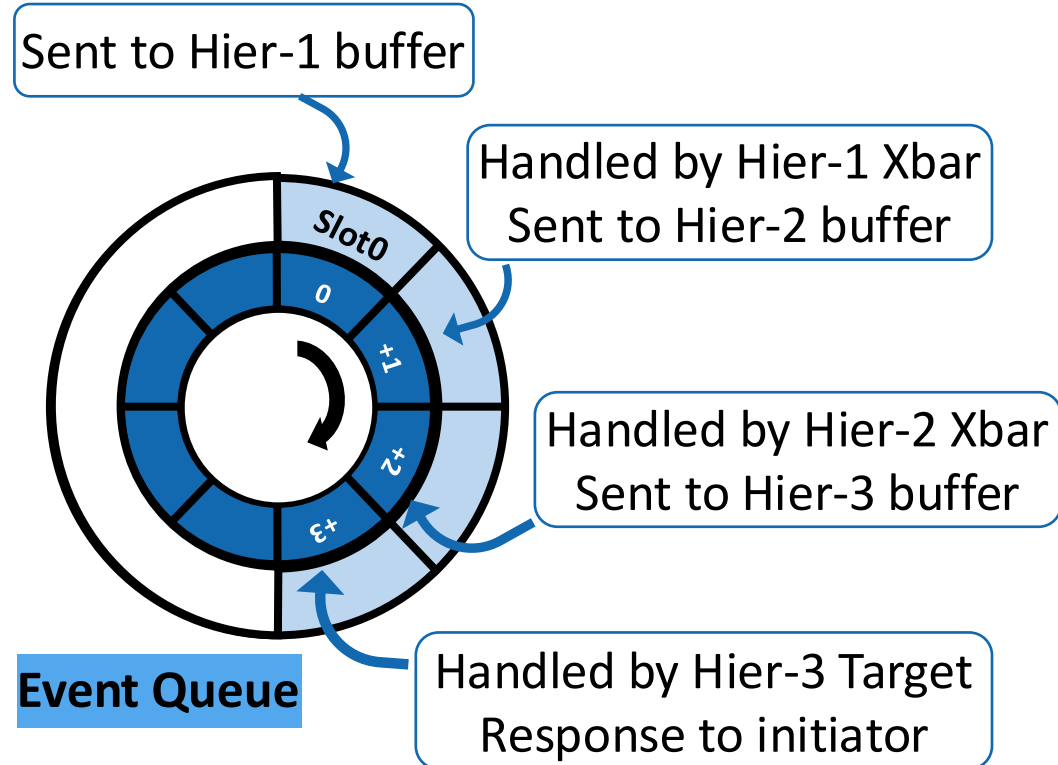
Benchmark	Runtime
Axy 385024	1m 28s
Dotp 385024	1m 28s
Matmul 512x256x512	7m 41s
Matmul 256x256x256	4m 1s
Matmul 256x128x256	2m 7s
Matmul 128x128x128	1m 38s

Asynchronous Modelling in GVSoC



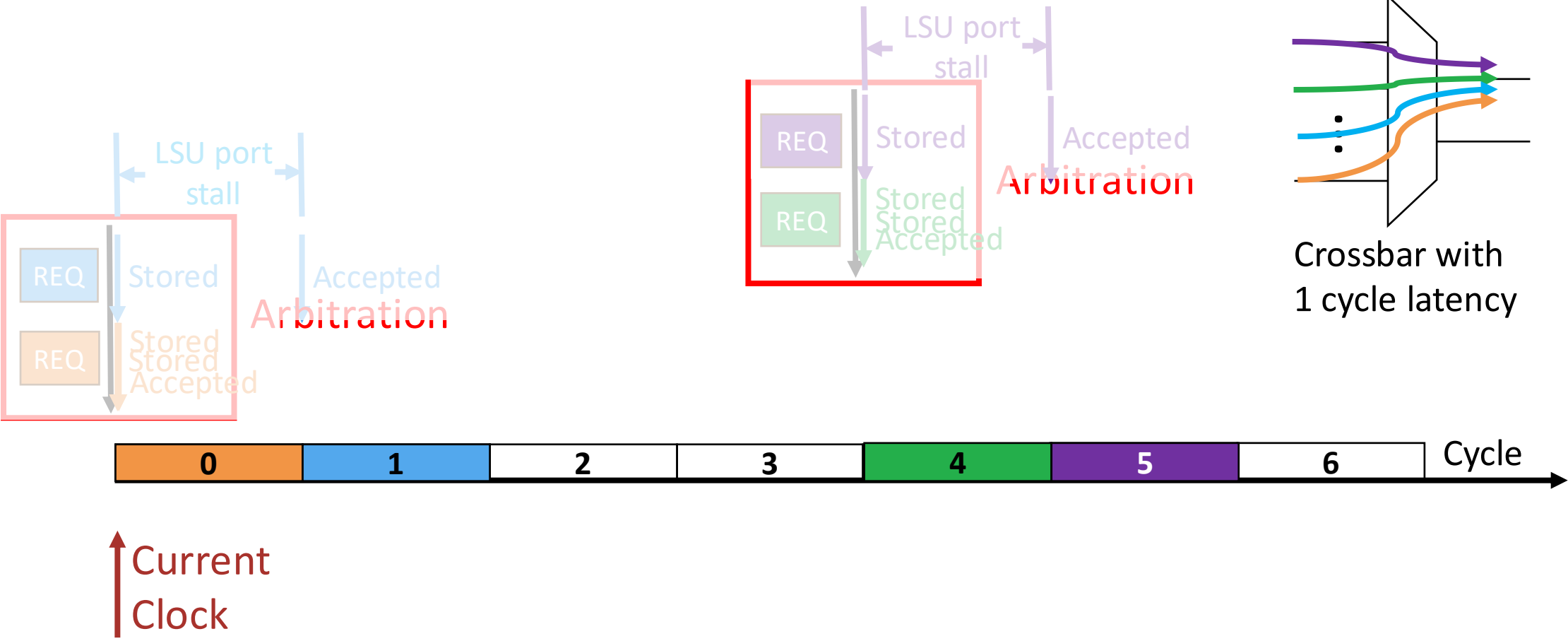
- **Requests propagate step by step**

- Stored in the buffer
- Get handled (then forward / return)

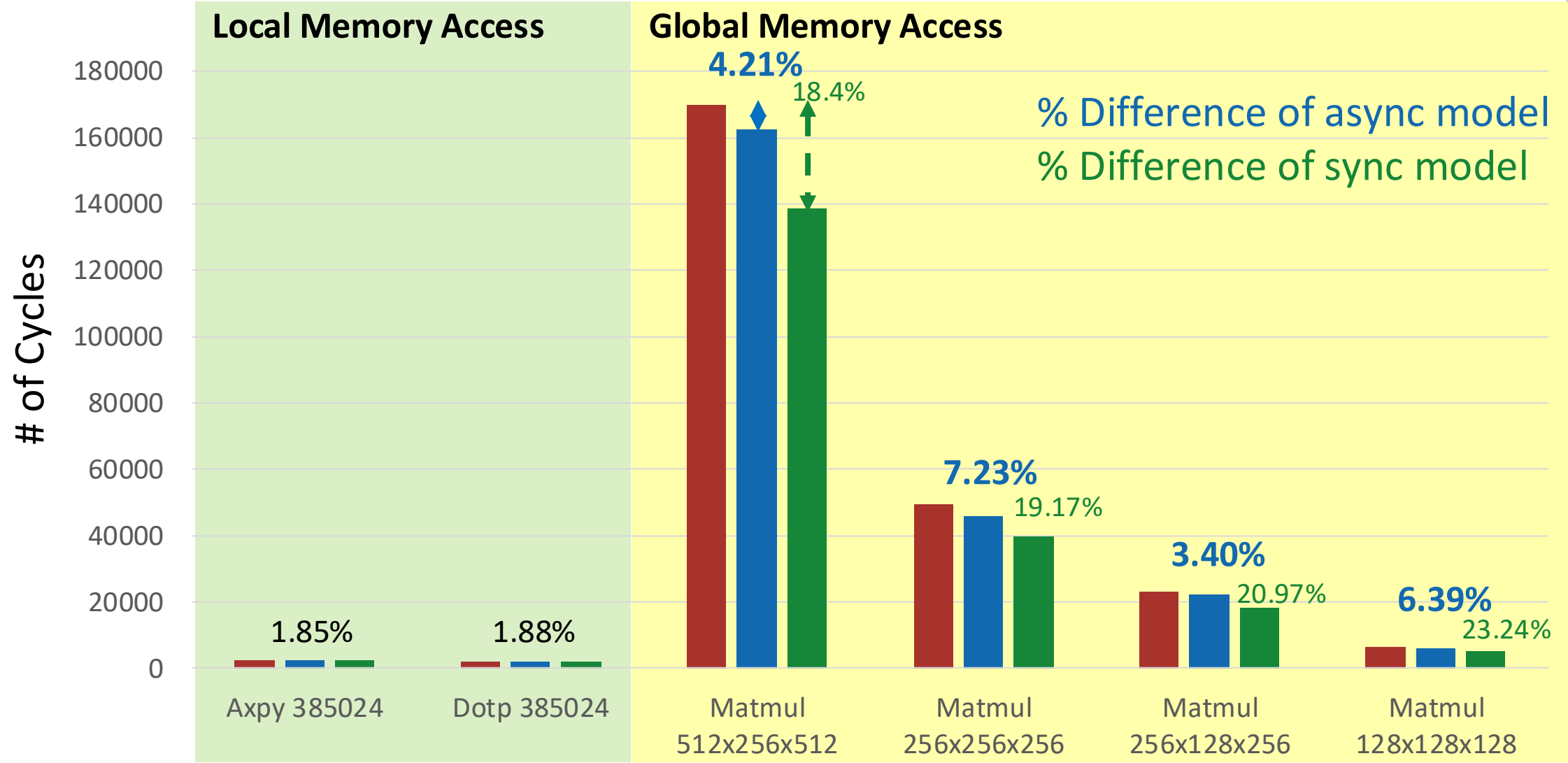


Stay on synchronous for response

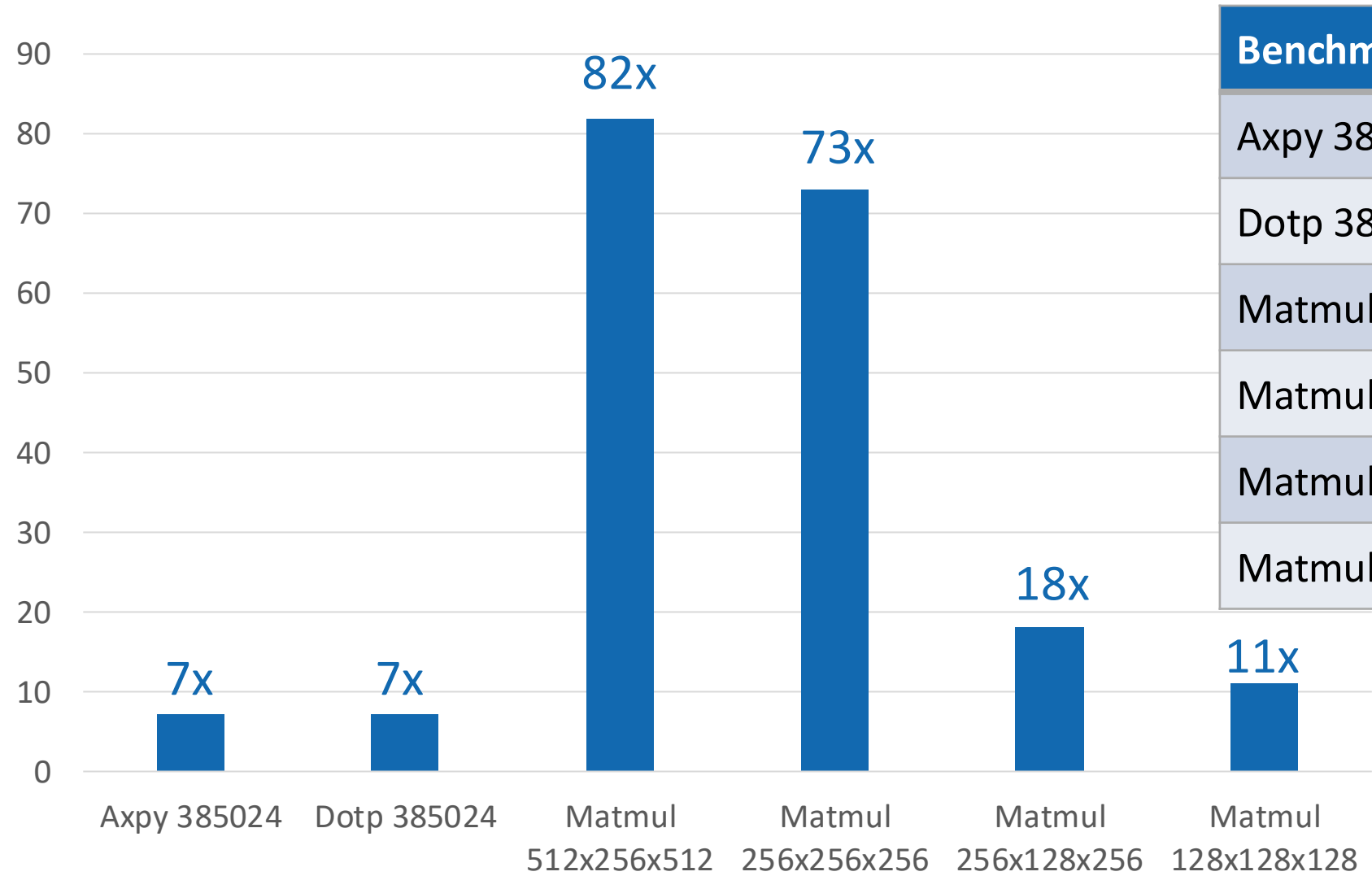
Asynchronous Modelling



Asynchronous Modelling in GVSoC

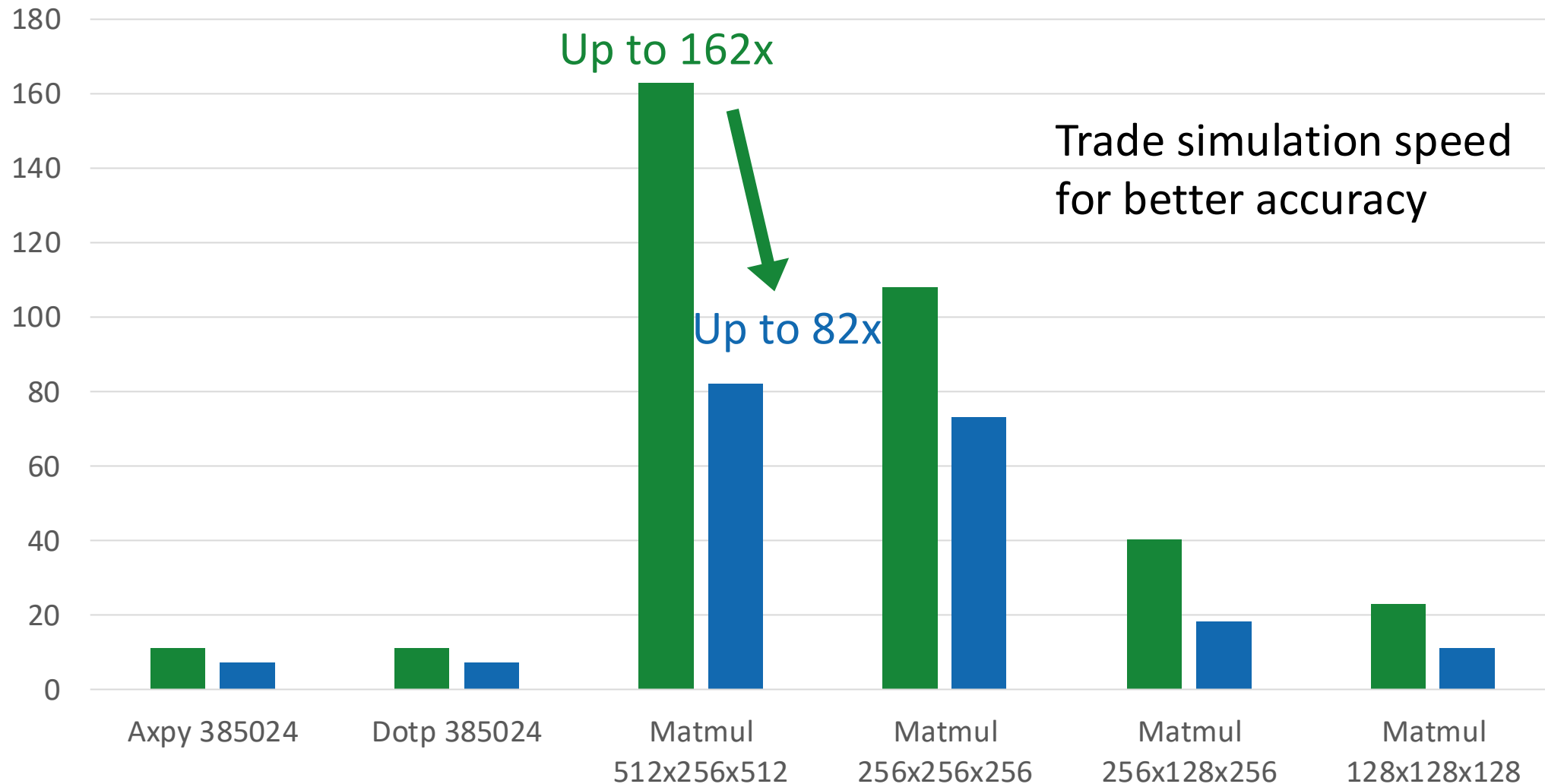


Speed up compared with RTL simulation



Benchmark	Runtime
Axdy 385024	2m 14s
Dotp 385024	2m 3s
Matmul 512x256x512	15m 15s
Matmul 256x256x256	6m 10s
Matmul 256x128x256	4m 43s
Matmul 128x128x128	3m 12s

Sync v.s. Async Simulation Speed

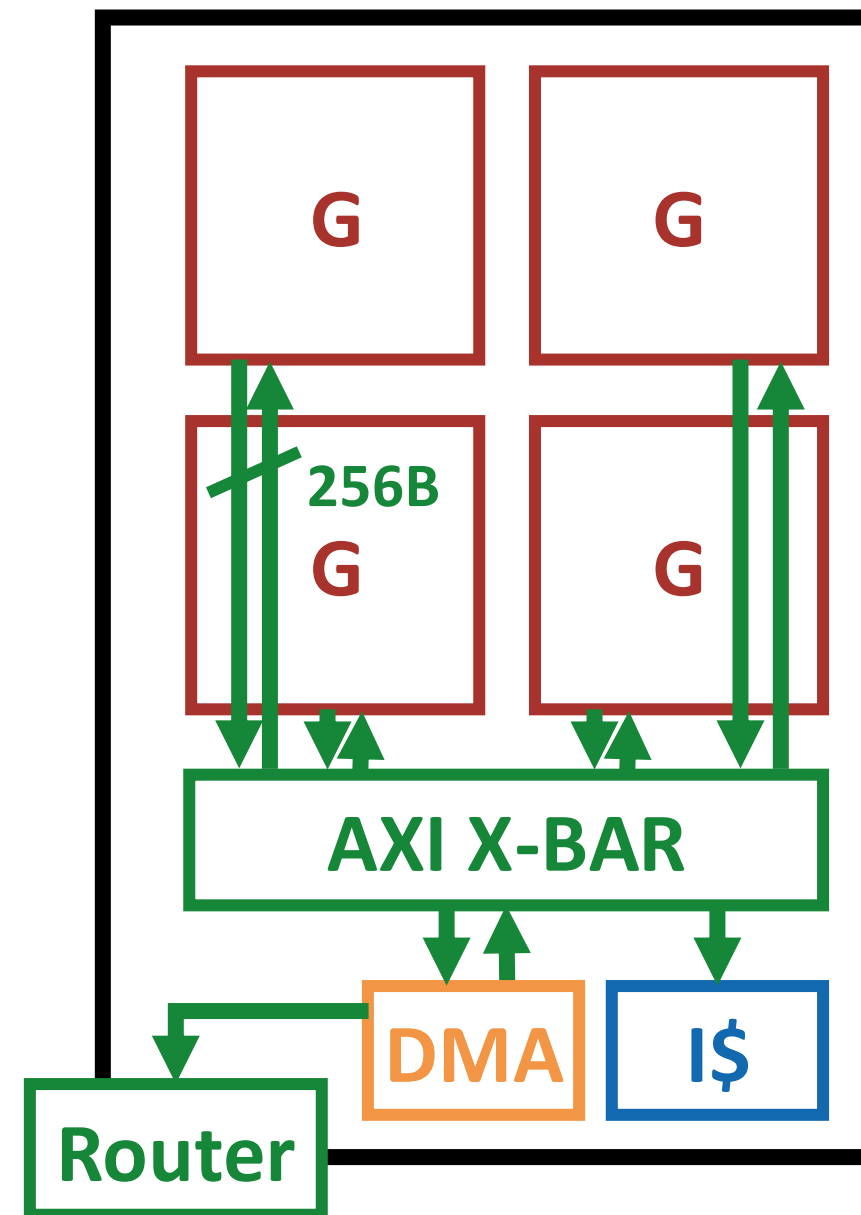
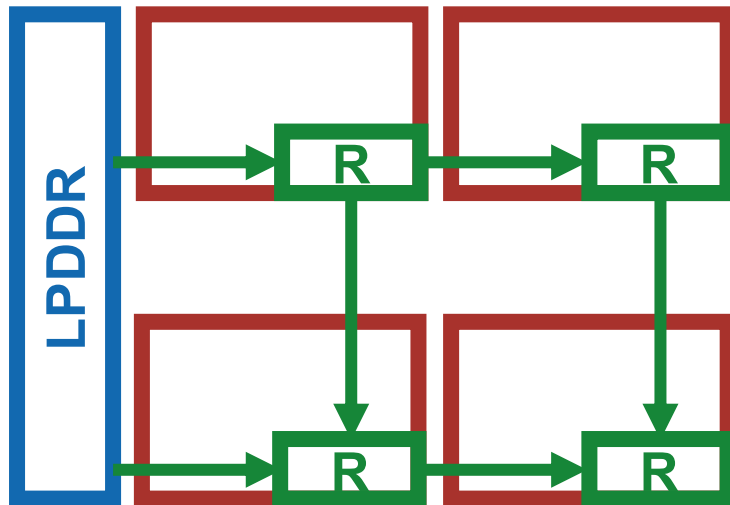




- **WIP on TensorPool model of GVSoC:**
 - Integrated RedMule in the Tile, synchronization between RedMule and Snitch
 - Asynchronous modeling, sending one 32b memory request at a time
- **But synchronous model best suited for fast evaluation:**
 - Syntetically captures the details of the burst transactions
 - Gives error ~20% on runtime cycles (acceptable for evaluation)

Multi-Cluster TensorPool

- DMA programmable by the cores
- 256B/cycle read&write BW per Group
- Router to access L1 in other Clusters
- I\$ “pre-heated” with the binary to be executed



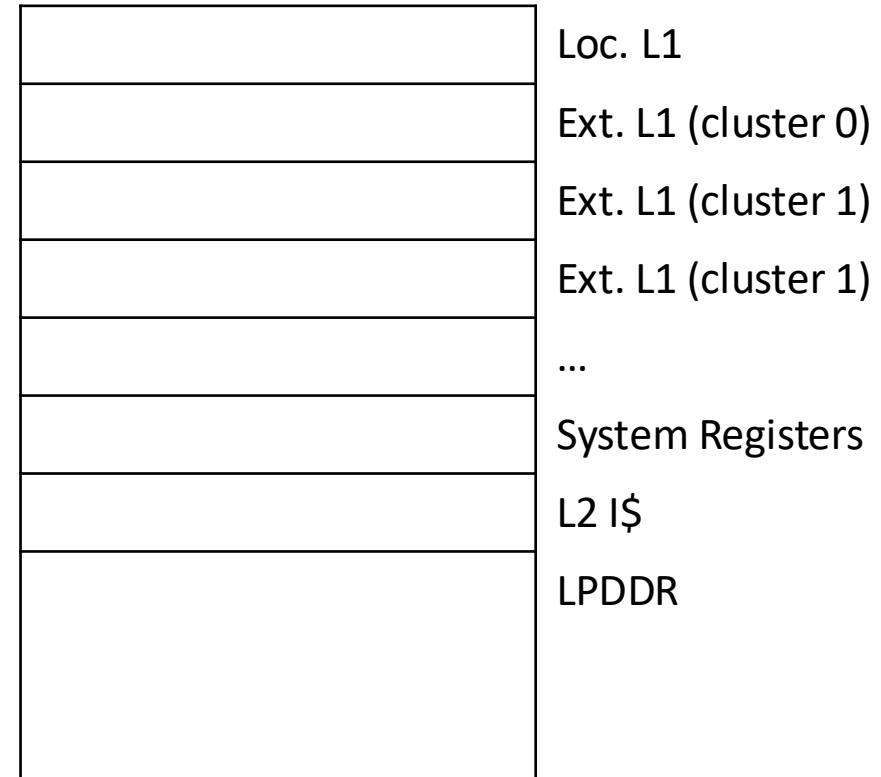


- **Ported from Softhier**

- Synchronization primitives
- DMA transfers (LPDDR to cluster and cluster to cluster)

- **Memory Map**

- Each cluster can address the local L1 both using the local and global addresses
- The L2 I\$ contains the same program for all clusters



Thank you!

